Tugas Komparasi Edge vs Cloud Computing pada Filtering Data IMU

Author: Erlanda Miko Prasetya
Teacher: Muhammad Qomaruz Zaman, S.T., M.T., Ph.D.
Class name: Sistem Berbasis Mikroprosesor

Tautan Terkait

GitHub: Repository GitHub
Video Report: Video Report

[1]. Pendahuluan

Data raw yang dihasilkan oleh sensor *Inertial Measurement Unit* (IMU) seperti MPU6050 umumnya memiliki tingkat *noise* yang tinggi. Untuk mendapatkan pembacaan akselerasi dan orientasi yang stabil, diperlukan algoritma penyaringan data digital. Salah satu pendekatan stokastik yang tangguh untuk sistem non-linier adalah *Particle Filter* (PF).

Namun, penerapan PF dengan jumlah partikel yang besar (misalnya 1000 partikel) menuntut beban komputasi yang masif. Laporan ini membahas komparasi dua arsitektur distribusi beban komputasi dalam ekosistem *Internet of Things* (IoT), yaitu Skenario A (*Cloud-Side Filtering*) dan Skenario B (*Edge-Side Filtering*). Transmisi data dari mikrokontroler (ESP32) menuju server (Raspberry Pi VM) dilakukan secara nirkabel memanfaatkan protokol UDP (*User Datagram Protocol*) untuk meminimalkan latensi jaringan. Mengingat limitasi ketersediaan perangkat keras fisik, pengujian ini diimplementasikan menggunakan metode *Software-in-the-Loop* (SITL) melalui rekaman log data sensor.

[2]. Metodologi dan Desain Sistem

[2.1]. Algoritma Particle Filter

Sistem menggunakan 1000 partikel untuk merepresentasikan distribusi probabilitas status sensor. Tahapan algoritma meliputi:

1. **Prediction:** Penambahan *process noise* secara acak pada setiap partikel.
2. **Update:** Penilaian bobot partikel berdasarkan probabilitas *Gaussian* terhadap nilai pengukuran sensor.
3. **Estimation:** Perhitungan rata-rata terbobot untuk menentukan estimasi akhir.
4. **Resampling:** Seleksi ulang partikel berdasarkan metode *Roulette Wheel* untuk menyingkirkan partikel berbobot rendah.

[2.2]. Skenario A: Cloud-Side Filtering

Pada arsitektur ini, node Edge (ESP32) berfungsi murni sebagai pengumpul data. ESP32 membaca data raw dari MPU6050 dan mentransmisikannya secara kontinyu via UDP pada frekuensi sekitar 50Hz. Raspberry Pi bertindak sebagai *server/cloud* yang menerima derau data tersebut dan mengeksekusi algoritma Particle Filter.

[2.3]. Skenario B: Edge-Side Filtering

Pada arsitektur ini, beban komputasi ditarik ke depan (*Edge*). ESP32 mengeksekusi iterasi 1000 partikel secara mandiri di dalam unit mikrokontrolernya. Data yang dikirimkan via UDP menuju Raspberry Pi adalah data yang sudah bersih (*filtered*), sehingga Raspberry Pi hanya bertugas untuk *logging* dan visualisasi antarmuka.

[3]. Hasil dan Analisis

[3.1]. Visualisasi dan Konvergensi Algoritma

Gambar 1 dan Gambar 2 menyajikan visualisasi data historis dari masing-masing skenario.

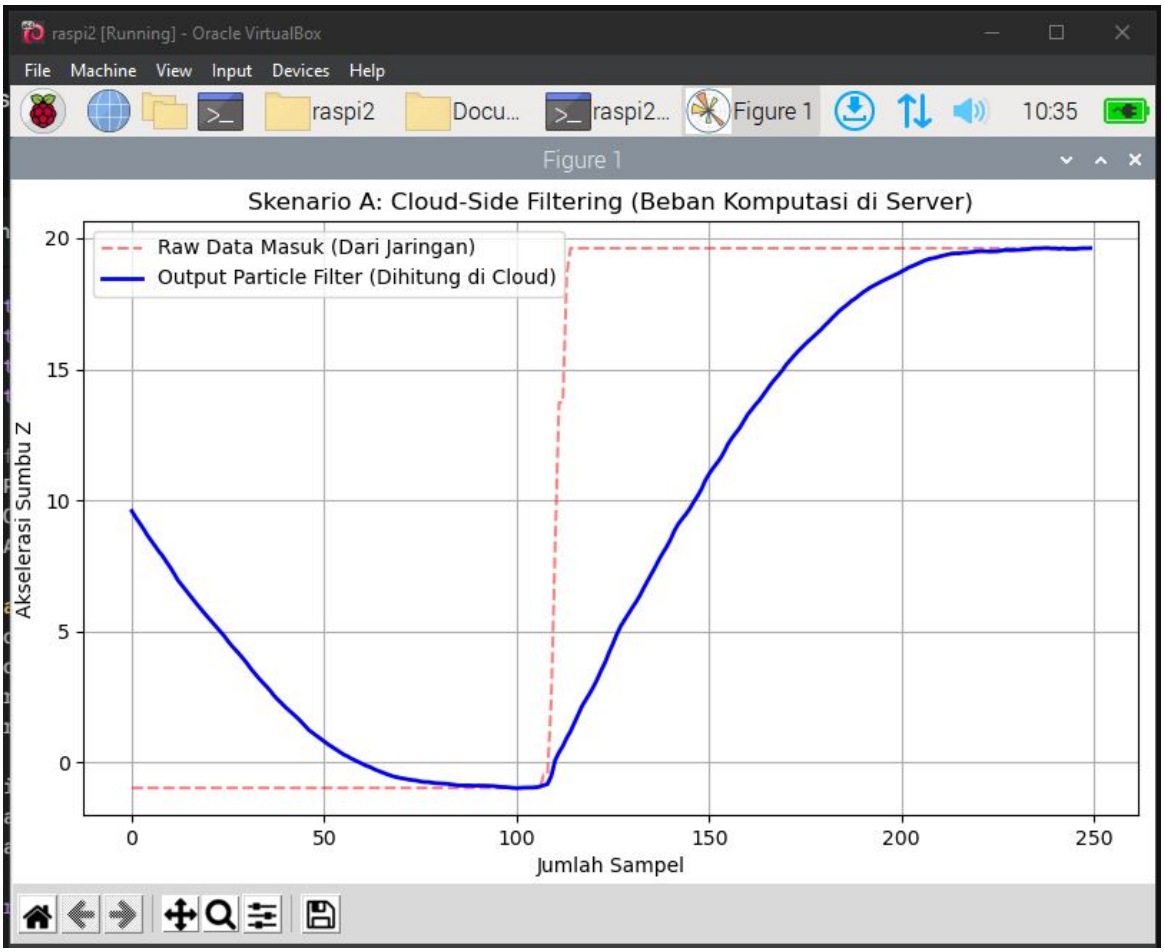

Figure 1: Komparasi Raw Data dan Filtered Data pada Skenario A.

Berdasarkan Gambar 1, dapat diamati beberapa fenomena kontrol kritis:

- **Initial Convergence:** Pada awal siklus (sampel 0 hingga 80), terdapat deviasi antara raw data (sekitar -1.0 m/s^2) dan output filter. Garis filter secara asimtotik menurun mendekati nilai riil, merepresentasikan proses *resampling* partikel dalam mencari *state* sebenarnya.
- **Noise Rejection:** Saat terjadi lonjakan data ekstrem pada sampel ke-105, filter tidak serta-merta mengikutinya secara instan, membuktikan ketahanan sistem terhadap anomali pembacaan sesaat (*outliers*).
- **Tracking Latency:** Sifat penghalusan sinyal ini menghasilkan efek inersia, di mana sistem membutuhkan waktu (jumlah sampel tertentu) untuk menyusul *step response* yang persisten.

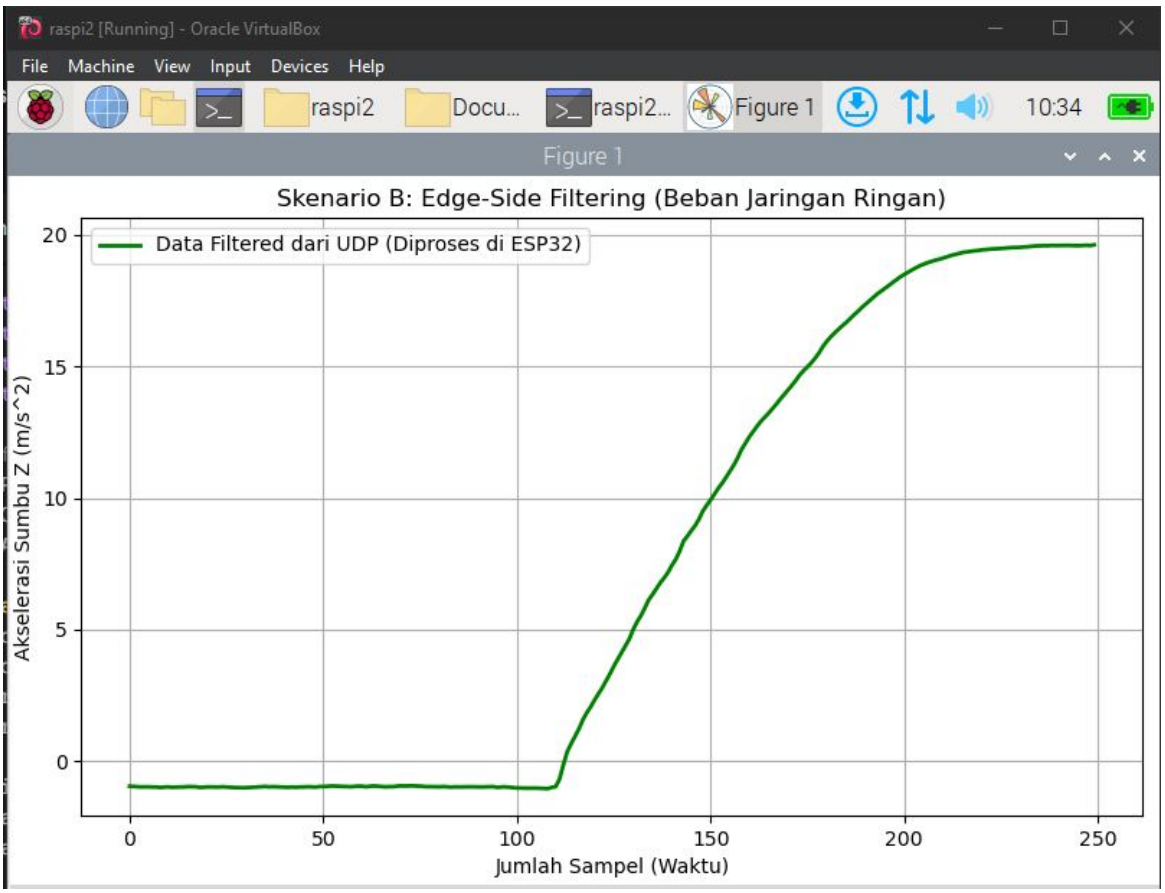

Figure 2: Data Filtered yang diterima oleh Raspberry Pi pada Skenario B.

Pada Gambar 2, kurva yang ditransmisikan dan diterima server telah berupa garis mulus, mengonfirmasi bahwa seluruh komputasi stokastik berhasil ditangani di level Edge.

[3.2]. Analisis Processing Time dan Trade-Off

Berdasarkan log eksekusi, terdapat perbedaan metrik kinerja yang mencolok:

- **Skenario A (Cloud):** Konsumsi waktu baca di ESP32 sangat minimal. Beban berat dilimpahkan ke Raspberry Pi yang mampu memproses kalkulasi probabilitas matriks jauh lebih efisien berkat arsitektur prosesor ARM berkecepatan tinggi. Arsitektur ini menuntut ketersediaan *bandwidth* yang tinggi secara konstan.
- **Skenario B (Edge):** Raspberry Pi menerima paket dengan waktu parsing yang terabaikan (sekitar 0.001 ms). Arsitektur ini sangat hemat *bandwidth*. Sisi negatifnya, kalkulasi dan distribusi probabilitas 1000 partikel memberi beban ekstrim pada siklus *clock* ESP32, berisiko mengganggu interval *sampling rate* sensor.

[4]. Kesimpulan

Implementasi Particle Filter dengan sampel partikel yang tinggi (1000 partikel) kurang optimal apabila ditugaskan sepenuhnya pada level *Edge* seperti mikrokontroler standar, karena dapat menyebabkan kendala *real-time processing*. Skenario A (*Cloud-Side Filtering*) terbukti menjadi arsitektur yang lebih reliabel untuk topologi kendali berbasis sensor intensif ini. Kinerja Skenario A mampu menjaga mikrokontroler tetap responsif mengumpulkan matriks data sekaligus memanfaatkan keunggulan komputasi *server* untuk mereduksi *noise* dinamis MPU6050.